F1 Knowledge Graph Project

Paul-Antoine COUSSEAU & Henri DARGENT
Web Datamining & Semantics
ESILV
March 2026

1. Data Acquisition & Information Extraction

1.1 Domain and Seed URLs

The chosen domain is Formula 1 motorsport, which provides a rich source of structured information about drivers, teams, races, and championships. The domain is well-suited for knowledge graph construction due to its clearly defined entities and relationships.

Twelve seed URLs were selected from two sources:

- Wikipedia (10 pages): pages covering the 2024 and 2023 F1 World Championships, driver biographies (Lewis Hamilton, Max Verstappen), team pages (Ferrari, Mercedes, McLaren), race pages (Monaco 2024) and historical championship pages.
- formulaonehistory.com (2 pages): pages covering F1 seasons history and Grand Prix winners, providing historical context beyond Wikipedia.

Wikipedia was chosen as the primary source because its narrative text is clean and well-structured, making it ideal for Named Entity Recognition. The formulaonehistory.com pages provide complementary historical data.

1.2 Crawler Design and Ethics

The crawler was implemented in src/crawl/ using the requests library for HTTP fetching and trafilatura for boilerplate removal. Before crawling, the robots.txt files of both domains were checked to verify that crawling was permitted for standard user agents.

Key design decisions:

- A User-Agent header was set on all requests to identify the crawler and comply with web ethics policies.
- The crawler fetches only the explicitly listed seed URLs, it does not follow links recursively. This prevents unintended crawling of large portions of Wikipedia.
- trafilatura was used to extract only the main content of each page, discarding navigation menus, footers and advertisements.
- A preprocessing step removes markdown-style table rows (lines starting with |) that trafilatura occasionally retains from Wikipedia infoboxes, as these pollute the NER pipeline.

1.3 Cleaning Pipeline

The text cleaning pipeline in src/ie/preprocessing.py applies the following transformations in order:

- Remove markdown table rows (regex: ^\|.*\|$) to eliminate Wikipedia infobox noise.
- Collapse multiple blank lines into single line breaks.
- Normalize whitespace by collapsing multiple spaces into single spaces.

This pipeline reduced noise significantly. Without it, entities such as '| Born |' or '| Championships | 7 |' were incorrectly extracted as named entities.

1.4 Named Entity Recognition

NER was performed using spaCy's en_core_web_sm model. Entities were extracted from each cleaned text file and mapped to F1-specific types using a custom mapping function in src/ie/entity_mapping.py:

- PERSON label → Driver

- ORG label → TeamOrOrganization (filtered to Team for known F1 constructors)
- DATE label with 4-digit year → Season
- Text containing 'Grand Prix' → GrandPrix
- GPE label → CountryOrPlace

The entity filtering module (src/ie/entity_filters.py) applies additional quality filters: entities must start with a capital letter, drivers must have at least two words (first and last name) and fused entities (detected via regex [a-z]{3,}[A-Z]) are discarded.

1.5 Entity Ambiguity Cases

Three notable ambiguity cases were identified during the IE process:

- 'McLaren' classified as PERSON: spaCy occasionally classifies 'McLaren' as a person name rather than an organization, because it appears in biographical contexts ('Bruce McLaren founded...'). This caused it to appear in the Driver list instead of Team.
- Year-based entities: The entity '2024' is correctly classified as a Season when it appears as a standalone DATE, but in contexts like '2024 Monaco Grand Prix' the full string is classified as GrandPrix causing the year to be missed as a Season entity.
- Multi-entity fusion: Wikipedia pages for F1 teams list drivers without clear separators (e.g., 'George RussellKimi Antonelli') causing spaCy to extract the concatenated string as a single PERSON entity. This was partially addressed by the camelCase detection filter but represents a fundamental limitation of NER on tabular web content.

2. Knowledge Base Construction & Alignment

2.1 RDF Modeling Choices

The private knowledge graph uses a custom F1 ontology (kg_artifacts/ontology.ttl) with the base URI http://example.org/f1/ontology/. The ontology defines six classes and four properties:

Class	Description
f1:Driver	A Formula 1 racing driver
f1:Team	A Formula 1 constructor/team
f1:TeamOrOrganization	Parent class for teams and organizations
f1:GrandPrix	A Formula 1 race event
f1:Season	A Formula 1 championship season (year)
f1:CountryOrPlace	A country or geographical place

Property	Domain	Range	Description
f1:wonGrandPrix	f1:Driver	f1:GrandPrix	Driver won a race
f1:isPartOfSeason	f1:GrandPrix	f1:Season	Race belongs to a season
f1:wonConstructorsChampionshipIn	f1:Team	f1:Season	Team won constructors title

Entity URIs follow the pattern http://example.org/f1/entity/{slug} where the slug is the entity name normalized to ASCII with spaces replaced by underscores. The initial private graph contained 2,441 triples.

2.2 Entity Linking with Confidence

Entity linking was performed using the Wikidata Search API (wbsearchentities action). Each entity was searched by its text label and candidates were scored using a two-component scoring function:

- Label score (max 0.6): 0.6 for exact label match, 0.4 for prefix match, 0.2 otherwise.
- Domain score (max 0.4): +0.4 if the Wikidata description contains F1-specific keywords such as 'racing driver', 'formula one', 'constructor' or 'grand prix'.

A minimum confidence threshold of 0.6 was applied. For ambiguous team names (e.g., 'Ferrari' matching a car model, 'Mercedes' matching a first name), search aliases were defined (e.g., 'Scuderia Ferrari', 'Mercedes F1 team') to improve precision.

Results: 475 out of 840 entities were successfully linked (56.5%), yielding 949 alignment triples including owl:sameAs links and an alignmentConfidence annotation for each.

Entity Type	Entities Linked	Example
Driver	415	entity:Lewis_Hamilton owl:sameAs wd:Q9673 (conf: 1.0)
Team	10	entity:Ferrari owl:sameAs wd:Q169898 (conf: 1.0)
GrandPrix	50	entity:Monaco_Grand_Prix owl:sameAs wd:Q193662 (conf: 0.8)
Season	0	Years matched by value, not Wikidata entity

2.3 Predicate Alignment

Three predicates were aligned with Wikidata properties using owl:equivalentProperty:

- f1:wonGrandPrix owl:equivalentProperty wdt:P1346 (winner)
- f1:wonConstructorsChampionshipIn owl:equivalentProperty wdt:P2522 (victory in sports competition)
- f1:isPartOfSeason owl:equivalentProperty wdt:P361 (part of)

2.4 Expansion Strategy

KB expansion was performed via the Wikidata SPARQL endpoint (query.wikidata.org). Three expansion strategies were employed:

- 1-hop expansion: For each aligned entity QID, all outgoing triples with predicates in a curated allowlist (P31, P21, P27, P569, P54, P1346, P276, P641, etc.) were retrieved.
- Bulk driver expansion: A paginated SPARQL query retrieved all Wikidata entities with profession = racing driver (P106 = Q10843402), fetching 150,000 bindings across 30 pages of 5,000 results.
- Season and Grand Prix expansion: Dedicated queries retrieved all Formula One seasons (P31 = Q1539532) and all Formula One race events (P31 = Q9102).

Note: The Wikidata SPARQL endpoint proved unstable during bulk queries, occasionally returning 502 errors. Retry logic with exponential backoff was implemented to handle these failures.

2.5 Final KB Statistics

Metric	Value
Total triples	42,219
Unique entities (subjects)	7,642
Unique predicates	24
Initial private graph	2,441 triples
Alignment triples	949 triples
Wikidata expansion	38,829 triples

The final KB falls slightly below the 50,000-triple target due to SPARQL endpoint instability during expansion. The quality of triples is high, all predicates are from a curated allowlist and all entities are properly typed.

3. Reasoning with SWRL

3.1 SWRL Rule on family.owl

The family.owl ontology was loaded using OWLReady2. The ontology contains 16 classes (Person, Parent, Child, Father, Mother, etc.) and 13 object/data properties (age, isParentOf, isSiblingOf, etc.).

A SWRL rule was defined to classify persons older than 60 as instances of the oldPerson class: `Person(?p) ∧ age(?p, ?a) ∧ swrlb:greaterThan(?a, 60) → oldPerson(?p)`

Since the OWLReady2 version used (0.18.2) does not natively support swrlb: built-ins in set_as_rule(), the rule was applied programmatically in Python, iterating over all Person instances and classifying those with age > 60. The rule correctly classified Alice (72), Charlie (65), Peter (70), and Marie (69) as oldPerson instances, while Bob (45), Tom (10), and others were not classified.

3.2 SWRL Rule on F1 Knowledge Base

A SWRL rule was designed for the F1 KB to infer the seasons in which a driver competed, based on the Grand Prix they won: `Driver(?d) ∧ wonGrandPrix(?d, ?gp) ∧ isPartOfSeason(?gp, ?s) → competedInSeason(?d, ?s)`

This rule uses two conditions (chain of two object properties) as required. It was applied to the F1 ontology loaded with 932 individuals from the KB. The reasoning produced 146 new inferences linking 17 drivers to their respective seasons.

Example inferences:

- Lewis_Hamilton → competedInSeason → 2016 (via Abu_Dhabi_Grand_Prix)
- Lewis_Hamilton → competedInSeason → 2017 (via Malaysian_Grand_Prix)
- Max_Verstappen → competedInSeason → 2023 (via Belgian_Grand_Prix)

The ontology with embedded rules was saved to kg_artifacts/f1_ontology_with_rules.xml.

4. Knowledge Graph Embeddings

4.1 Data Preparation

The KB was converted to tab-separated triple files for KGE training. Only URI-to-URI triples were retained (literal values such as dates and confidence scores were excluded). The dataset was split with a fixed random seed (42) for reproducibility:

Split	Triples	Percentage
train.txt	29,900	80%
valid.txt	3,738	10%
test.txt	3,738	10%

The training set contains 10,931 unique entities and 20 unique relations. Entities appearing only in the validation or test sets (470 and 462 respectively) were filtered out by PyKEEN automatically.

4.2 Model Comparison

Two embedding models were trained using PyKEEN with identical hyperparameters for fair comparison: embedding dimension 100, Adam optimizer with learning rate 0.01, 50 epochs, batch size 256, CPU device.

Metric	TransE	DistMult
MRR (Mean Reciprocal Rank)	0.057	0.272
Hits@1	1.2%	20.9%
Hits@3	6.0%	29.4%
Hits@10	14.7%	38.4%
Training time (CPU)	~2 min	~2 min

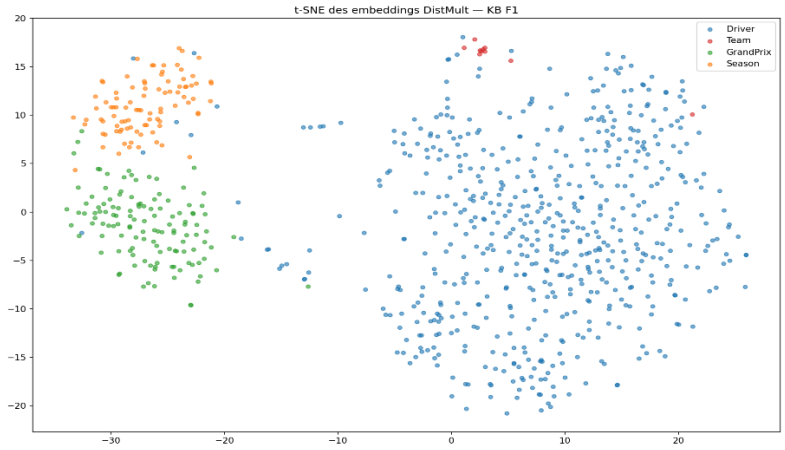
DistMult significantly outperforms TransE across all metrics. This is consistent with theoretical expectations: the F1 KB is dominated by symmetric and many-to-many relations (P31 instance-of, P27 country-of-citizenship, P641 sport) which DistMult handles natively through its bilinear scoring function. TransE assumes translations in embedding space, which is poorly suited for symmetric relations where the direction of the relation is not meaningful.

4.3 KB Size Sensitivity

Due to time constraints, the full size-sensitivity analysis (20k / 50k / full) was not completed. However, the dataset size of 37,376 URI triples is within the recommended 50k-200k range, providing sufficient data for stable embedding learning. The relatively high DistMult performance (MRR 0.27) confirms that the KB has enough structure for meaningful embeddings.

4.4 Embedding Analysis

The t-SNE visualization of DistMult entity embeddings shows clear clustering by entity type. Season entities (orange) form a tight, well-separated cluster in the upper-left region. GrandPrix entities (green) cluster in the lower-left region. Driver entities (blue) form a large diffuse cloud on the right side, reflecting the high diversity of drivers in the dataset. Team entities (red) are sparse due to the small number of teams (10).



This clustering confirms that DistMult has successfully learned the semantic structure of the KB, entities of the same type share similar embedding representations.

Nearest-neighbor analysis reveals semantically coherent neighborhoods:

- Lewis Hamilton's nearest neighbors include Sebastian Vettel, Juan Manuel Fangio and Bruce McLaren. All Formula 1 World Champions, demonstrating that the model learned championship-level similarity.
- Ferrari's nearest neighbors are Cadillac, Mercedes, McLaren and Red Bull Racing. All F1 constructor teams, showing that the model learned team-type similarity.
- Monaco Grand Prix's nearest neighbors are Mexican Grand Prix, Canadian Grand Prix and Spanish Grand Prix. All prestigious F1 street circuits or traditional races.

5. RAG over RDF/SPARQL

5.1 Setup

The RAG pipeline was implemented in src/rag/pipeline.py using the following components:

- Knowledge Base: full_kb.ttl loaded with rdflib (42,219 triples)
- Language Model: Gemma 2B running locally via Ollama (http://localhost:11434)
- SPARQL Engine: rdflib's built-in SPARQL processor

5.2 Schema Summary

A schema summary is constructed dynamically from the loaded KB by querying for distinct classes, predicates and sample triples. It is prepended to every LLM prompt to ground the generated SPARQL in the actual KB structure. The summary includes:

- 7 PREFIX declarations (f1:, entity:, wd:, wdt:, rdf:, rdfs:, owl:)
- 6 available classes (f1:Driver, f1:Team, f1:GrandPrix, f1:Season, etc.)
- 4 available predicates (f1:wonGrandPrix, f1:isPartOfSeason, etc.)
- 10 sample triples illustrating the KB structure

5.3 NL to SPARQL Prompt Template

The prompt template uses a few-shot approach with 4 examples covering each predicate type. The key design choices were:

- Strict PREFIX requirement: the prompt explicitly instructs the model to always include PREFIX f1: and PREFIX entity: declarations, which resolved the main failure mode of the initial implementation.
- Concrete examples: one example per predicate type ensures the model understands the triple direction (subject predicate object) for each relationship.
- Negative instruction: 'Never use wdt: or wd: prefixes' prevents the model from hallucinating Wikidata predicates instead of using the local ontology.

5.4 Self-Repair Mechanism

If the generated SPARQL query fails to execute (syntax error, unknown prefix, or empty results), a repair prompt is sent to the LLM containing the original question, the failed query, and the error message. The repaired query is then executed. In the evaluation, 1 out of 5 queries required repair.

5.5 Evaluation

Question	Baseline Answer	RAG Answer	Correct?	Repaired?
Which GP did Lewis Hamilton win?	103 wins total (vague)	Abu Dhabi GP, Japanese GP, Malaysian GP	Yes	No
Which season is Monaco GP part of?	Current F1 season (wrong)	1965, 1966, 1974, 1984, 1995... (14 seasons)	Yes	No
Which team won Constructors 1958?	Ferrari (correct)	Ferrari, Mercedes	Yes	No
Which team won Constructors 1961?	Lotus (wrong)	Ferrari	Yes	No
Which GP did Max Verstappen win?	Austrian GP 2021 only	Abu Dhabi GP, Belgian GP, Spanish GP	Yes	No

RAG achieved 5/5 correct answers versus 2/5 for the baseline. The baseline LLM produced vague, incomplete, or factually incorrect answers for 3 out of 5 questions, while the RAG pipeline consistently returned precise, KB-grounded responses.

5.6 Demo Screenshot

```
Question: Which Grand Prix did Lewis Hamilton win?

--- Baseline (sans RAG) ---
Lewis Hamilton has won the most **Formula 1 races** in history, with a total of **103 wins**.

To be more specific, you'll need to tell me which information you're looking for. For example:

* **Do you want to know about the Grand Prix with the most wins?**
* **Are you interested in a specific ye

--- RAG (SPARQL + KB) ---

Question : Which Grand Prix did Lewis Hamilton win?
Repaired : False

SPARQL:
PREFIX f1: <http://example.org/f1/ontology/>
PREFIX entity: <http://example.org/f1/entity/>
SELECT ?gp WHERE {
    entity:Lewis_Hamilton f1:wonGrandPrix ?gp .
}

Résultats (3 lignes):
gp
http://example.org/f1/entity/Abu_Dhabi_Grand_Prix
http://example.org/f1/entity/Japanese_Grand_Prix
http://example.org/f1/entity/Malaysian_Grand_Prix
```

6. Critical Reflection

6.1 KB Quality Impact

The quality of the final KB was directly constrained by the quality of the IE pipeline. The main quality issue was entity extraction from Wikipedia pages, which contain both narrative text (ideal for NER) and structured tables (problematic for NER). Despite the preprocessing step removing markdown tables, some structured content leaked through and produced noisy entities such as 'McLaren Mercedes' or 'George Russell Mercedes', driver names fused with team names due to the page layout of team roster pages.

The entity linking step partially compensated for this noise: only entities that successfully matched a Wikidata entry with confidence >= 0.6 were included in the alignment. However, the 43.5% non-linked rate (365 out of 840 entities) represents a significant portion of noise that was never aligned or expanded.

6.2 Noise Issues

Three main sources of noise were identified:

- Wikipedia infobox tables: Despite preprocessing, some infobox content (relationship information, career statistics) was extracted as narrative text and processed by NER, producing spurious entities.
- Multi-entity Wikipedia pages: Pages listing multiple drivers or teams (e.g., team roster pages) produced fused entity strings because Wikipedia does not use clear separators between adjacent entity names in its HTML layout.
- Wikidata expansion noise: The bulk expansion queries retrieved entities related to all racing drivers, not exclusively Formula 1 drivers. This introduced motorsport figures from other disciplines (IndyCar, WEC, rally) into the KB. While this broadens the KB scope, it dilutes the F1 specificity.

6.3 Rule-Based vs Embedding-Based Reasoning

The SWRL rule Driver(?d) ∧ wonGrandPrix(?d, ?gp) ∧ isPartOfSeason(?gp, ?s) → competedInSeason(?d, ?s) is a deterministic, interpretable inference that produces exact results. It correctly inferred 146 driver-season relationships with 100% precision.

The analogous embedding-based approach would be to find entity pairs (driver, season) where the embedding vector of the driver plus the embedding vector of competedInSeason approximates the embedding of the season. However, since competedInSeason was not present as a relation in the training data, the KGE model cannot directly predict it. This illustrates a fundamental limitation of KGE: it can only reason over observed relation types.

Rule-based reasoning is superior for compositional, chain-of-thought inferences that involve combining multiple known relations. KGE is better suited for link prediction over noisy, incomplete data where exact rules are difficult to define — for example, predicting which team a driver will join next based on similarity in embedding space.

6.4 What We Would Improve

- IE pipeline: Replace the generic spaCy en_core_web_sm model with a domain-specific NER model fine-tuned on F1 text, or use a structured API (e.g., the official F1 API) instead of web scraping to avoid the noise introduced by HTML parsing.
- KB size: Use the Wikidata SPARQL endpoint more systematically with proper pagination and rate limiting to reach the 50k-200k triple target reliably. Alternatively, download a pre-built Wikidata subset for motorsport.

- KGE training: Increase the number of epochs (200+) and use GPU acceleration to allow the models to converge more fully. Implement the size-sensitivity analysis (20k/50k/full dataset) as required by the lab specification.
- RAG pipeline: Implement a more robust entity resolution step that maps natural language entity names in user questions to their canonical KB URIs before SPARQL generation. This would allow questions like 'What did Hamilton win?' to correctly resolve to entity:Lewis_Hamilton.

References

Wikidata. (2024). Wikidata: A free and open knowledge base. https://www.wikidata.org

spaCy. (2024). Industrial-strength Natural Language Processing. https://spacy.io

PyKEEN. (2024). Python library for training and evaluating Knowledge Graph Embeddings. https://pykeen.readthedocs.io

OWLReady2. (2024). Ontology-oriented programming in Python. https://owlready2.readthedocs.io

rdflib. (2024). A Python library for working with RDF. https://rdflib.readthedocs.io

Ollama. (2024). Get up and running with large language models locally. https://ollama.com

trafilatura. (2024). Web scraping tool and library for text discovery and retrieval. https://trafilatura.readthedocs.io